

Dokumentacja modułu regresji liniowej

Dominik Lech

7 kwietnia 2026

1 Założenia projektu

Zadaniem mojej części projektu jest wystawić endpoint HTTP, który pozwoli na trenowanie modelu regresji liniowej na podstawie zadanych plików CSV na różne sposoby – np. z dopuszczeniem różnych automatycznych transformacji, regularyzacja L1 i L2, ze sprawdzeniem wytrenowanego modelu na zbiorze testowym i tak dalej. Mój moduł zapewnia osobie (np. analitykowi) korzystającej z programu opcję zobaczenia, które kolumny są dobrymi predyktorami dla wybranej kolumny, a które są mniej istotne, jak prosty model działa na zbiorze testowym i jaka część wariancji wyjaśnia (wartość R^2).

Projekt zakłada, że dane w CSV są już odpowiednio przygotowane i oczyszczone – tutaj tylko trenuję regresję liniową dla wybranych kolumn z podanego pliku. Dlatego pliki CSV nie mogą mieć brakujących wartości ani zmiennych jakościowych – one powinny być wcześniej zmienione na ilościowe, np. poprzez kodowanie zero-jedynkowe (one-hot encoding).

Dla `method="ols"`, zależnie od celu, zalecana jest standaryzacja danych przed podaniem ich do trenowania – tak żeby miało sens porównywanie wielkości współczynników przy predyktorach. Dla ridge i lasso standaryzacja jest **konieczna**, żeby algorytmy mogły zadziałać poprawnie.

2 Wykorzystane biblioteki

Wszystkie potrzebne biblioteki dodano do `requirements.txt`. Najważniejsze:

- `pandas` – do załadowania CSV do Pythona i zarządzania danymi w postaci ramki danych,
- `numpy` – do transformacji,
- `statsmodels` – do trenowania OLS,
- `scikit-learn` – do trenowania ridge i lasso.

3 Struktura projektu

Wszystko znajduje się w katalogu `regresje`, który zawiera:

- `linear_regression.py` – funkcja wywołująca inne funkcje i zwracająca końcowy wynik,
- `validations.py` – odpowiada za walidację danych,
- `apply_transformations.py` – aplikuje transformacje (dodaje nowe kolumny na podstawie istniejących),
- `ols.py` – trenuje model standardowej regresji liniowej,
- `lasso.py` – trenuje model z regularyzacją L1,

- `ridge.py` – trenuje model z regularyzacją L2,
- `utils.py` – małe pomocnicze funkcje,
- `dokumentacja/` – katalog zawierający dokumentację.

4 Dane wejściowe

4.1 Wymagane pola

- `train_file` – plik, na którym będziemy trenować model.
- `target_column` (string) – nazwa kolumny, którą model będzie przewidywać.
- `mode` (string) – tryb wyboru predyktorów:
 - `"all_parameters"` – wszystkie kolumny oprócz `target_column` są predyktorami,
 - `"include_parameters"` – tylko kolumny podane w `columns` są predyktorami,
 - `"exclude_parameters"` – wszystkie kolumny oprócz `target_column` i podanych w `columns` są predyktorami.

4.2 Pola opcjonalne

- `columns` (string) – lista nazw kolumn oddzielona przecinkami (używana dla `include_parameters` lub `exclude_parameters`).
- `transformations` (string) – lista transformacji w formacie JSON:

```
[{"column": "NAZWA_KOLUMNY", "type": "1/x"},
 {"column": "NAZWA_KOLUMNY", "type": "square"}]
```

Dostępne typy: `"1/x"`, `"square"`, `"log"`.

- `test_file` – opcjonalny plik testowy (musi mieć identyczne kolumny i typy jak `train_file`). Jeśli podany, po trenowaniu model jest testowany na nowych danych.
- `method` (string) – metoda trenowania:
 - `"ols"` – zwykła regresja liniowa,
 - `"lasso"` – regresja z regularyzacją L1 (współczynniki mogą spaść do zera),
 - `"ridge"` – regresja z regularyzacją L2 (współczynniki nie zerują się).
- `alpha` (float) – dodatni parametr regularyzacji, wymagany dla `lasso` i `ridge`. Im bliższy zeru, tym regularyzacja mniejsza; `alpha=0` jest blokowane – wtedy należy użyć `method="ols"`.

5 Dane wyjściowe

Przykładowa odpowiedź JSON:

```
{
  "parameters": [
    {
      "name": "intercept",
      "coefficient": "0.373785",
      "p_value": "0.000000",
      "standard_error": "0.019497",
      "conf_int_lower": "0.335488",
      "conf_int_upper": "0.412083"
    },
    ...
  ],
  "r_squared": "0.419415",
  "adj_r_squared": "0.409010",
  "f_p_value": "0.000000",
  "prediction_results": {
    "mse": "0.162862",
    "r2": "0.339213"
  }
}
```

Opisy pól:

- **parameters** – lista estymowanych współczynników.
 - **name** – nazwa parametru (lub **intercept** dla wyrazu wolnego).
 - **coefficient** – wartość współczynnika.
 - **p_value** – poziom istotności (poniżej 0,05 oznacza statystycznie istotny predyktor). Dla ridge/lasso – **null**.
 - **standard_error** – błąd standardowy estymatora. Dla ridge/lasso – **null**.
 - **conf_int_lower** / **conf_int_upper** – dolna i górna granica 95% przedziału ufności. Dla ridge/lasso – **null**.
- **r_squared** – R^2 na zbiorze treningowym (część wariancji wyjaśnionej przez model).
- **adj_r_squared** – skorygowany R^2 (kara za liczbę predyktorów).
- **f_p_value** – p-wartość testu F dla całego modelu. Dla ridge/lasso – **null**.
- **prediction_results** (obiekt, dostępny tylko jeśli podano **test_file**):
 - **mse** – średniokwadratowy błąd predykcji na zbiorze testowym.
 - **r2** – R^2 na zbiorze testowym. Jeśli jest zauważalnie mniejsze niż R^2 treningowe, sugeruje to overfitting.

Dla metod **lasso** lub **ridge** część pól (**p_value**, **standard_error**, **conf_int**, **f_p_value**) będzie miała wartość **null** – jest to oczekiwane.

6 Kody błędów (status 400 Bad Request)

- **INVALID_DATASET** – **train_df** jest pusty lub ma mniej niż 2 kolumny.
- **INVALID_MODE** – podano nieobsługiwany tryb **mode**.

- TARGET_COLUMN_NOT_FOUND – brak kolumny `target_column` w pliku treningowym.
- TEST_DATASET_BAD_STRUCTURE – `test_file` ma inne kolumny lub typy niż `train_file`.
- COLUMNS_REQUIRED – dla trybu `include_parameters` lub `exclude_parameters` brakuje parametru `columns`.
- UNKNOWN_COLUMNS – w `columns` podano nazwy kolumn nieistniejące w `train_file`.
- NO_FEATURE_COLUMNS_SELECTED – po przetworzeniu nie wybrano żadnej kolumny predyktorów.
- NON_NUMERIC_COLUMNS_NOT_ALLOWED – wykryto kolumnę nienumeryczną.
- NANS_NOT_ALLOWED – wykryto brakujące wartości (NaN).
- INVALID_TRANSFORMATIONS – niepoprawna struktura JSON dla transformacji.
- UNKNOWN_TRANSFORMATION_COLUMN – transformacja dotyczy kolumny nieistniejącej w `train_file`.
- LOG_TRANSFORMATION_NOT_ALLOWED_FOR_NON_POSITIVE_VALUES – logarytm z wartości niedodatnich.
- 1/x_TRANSFORMATION_NOT_ALLOWED_FOR_NEGATIVE_VALUES – dzielenie przez zero lub ujemne (wymagane dodatnie).
- UNKNOWN_TRANSFORMATION_TYPE – nieznany typ transformacji.
- POSITIVE_ALPHA_REQUIRED – `alpha` nie jest liczbą dodatnią.
- UNKNOWN_METHOD – podano nieobsługiwaną metodę.
- COULD_NOT_READ_CSV – nie udało się wczytać `train_file`.
- COULD_NOT_READ_TEST_CSV – nie udało się wczytać `test_file`.

7 Przykładowe wywołanie curl

```
curl --location 'http://127.0.0.1:8000/regressions/linear' \
--form 'mode="all_parameters"' \
--form 'columns="Fare,SibSp,Parch"' \
--form 'target_column="Survived"' \
--form 'train_file=@"sciezka/train.csv"' \
--form 'test_file=@"sciezka/test.csv"' \
--form 'transformations="[{"column": "Age", "type": "1/x"}, {"column": "Age", "type": "square"}]"' \
--form 'method="ridge"' \
--form 'alpha="0.01"'
```

8 Podsumowanie

Podsumowując, udało mi się utworzyć użyteczne narzędzie pozwalające na wygodne sprawdzanie rezultatów trenowania różnych modeli regresji liniowych w różnych scenariuszach na własnych danych, bez konieczności pisania własnego kodu. Mój moduł będzie wymagał interfejsu graficznego, który będzie łatwy do zaimplementowania dzięki odpowiedniemu opisaniu danych wejściowych, wyjściowych oraz kodów błędów.